

RMF Traffic Editor

GUI for creating 2D floor plans

TRAFFIC EDITOR



Why start with Traffic-Editor?

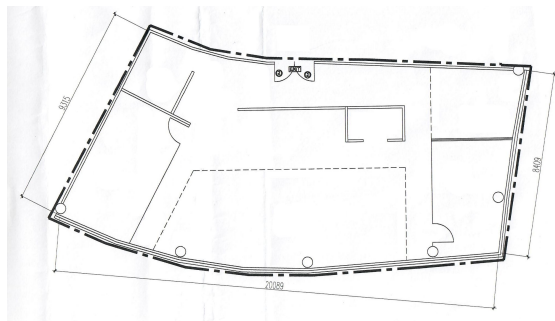
- When starting a new robot deployment, many important questions need to be considered and quantified:
 - Where will the robots operate?
 - How many fleets are there?
 - How the fleets interact?
 - How and where will humans interact with the robots?
- Typically it is important to start with a simulation
 - Large facilities can't be "paused" for experiments
 - Mistakes in simulators are much cheaper than in reality!
- Traffic Editor will help you set up a simulation and robot fleets

Start using Open-RMF in three easy steps

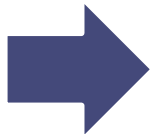
1. Find or create a building map
2. Create a simulation model
3. Make a robot traffic map

Step 1 may require human interaction.

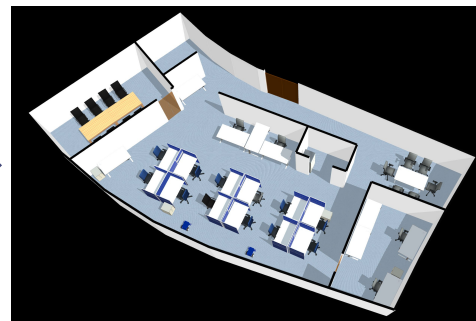
Traffic Editor helps you with steps 2 and 3.



Building floorplan



Annotated in *traffic_editor*



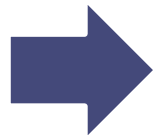
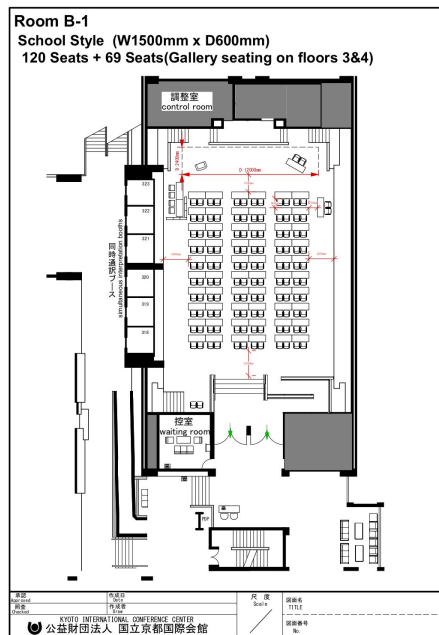
Physics-based Gazebo
simulation world

Step 1: get ready to import a map

Ideally, you'll start with an architectural drawing.

Example: the ROSCon 2022 venue provides a nice PDF of this workshop's room:

https://www.icckyoito.or.jp/planner/hall_and_room/room_b1/

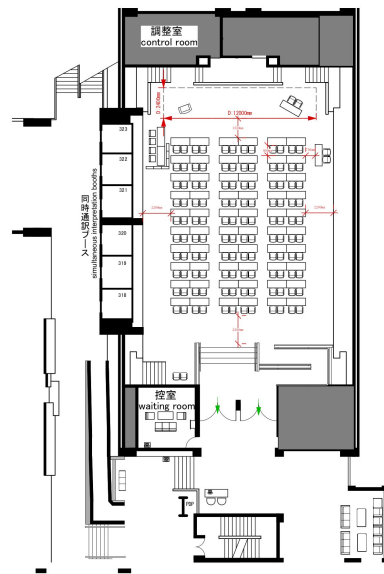
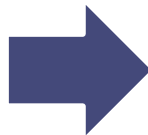


1. Rasterize the PDF
2. Crop borders away
3. Save as PNG

We typically use GIMP

<https://www.gimp.org/>

Goal: "decent" resolution,
typically a few megapixels



Rasterized PNG image

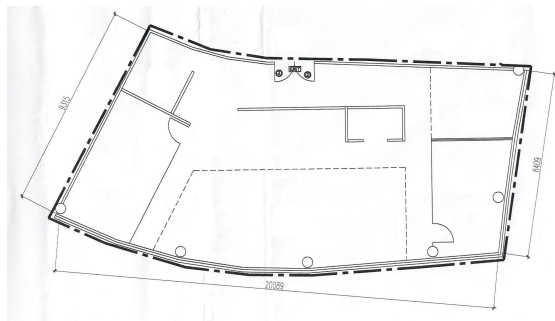
Floorplan (typically PDF)

Start using Open-RMF in three easy steps

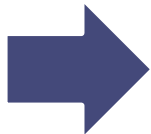
1. Find or create a building map 
- 2. Create a simulation model**
3. Make a robot traffic map

Step 1 may require human interaction.

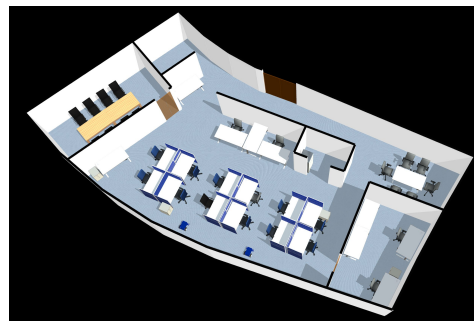
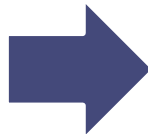
Traffic Editor helps you with steps 2 and 3.



Building floorplan



Annotated in *traffic_editor*



Physics-based Gazebo
simulation world

Step 2a(i): create a traffic editor project

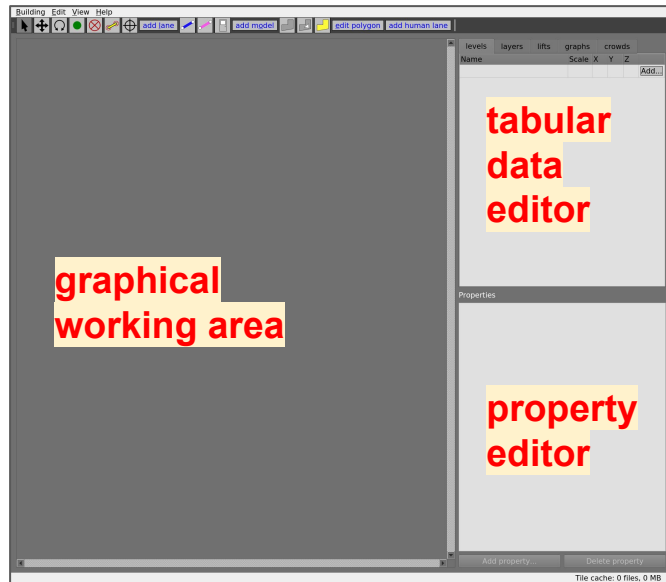
- [create a new directory named "buildings"]
- [copy the map PNG to the "buildings" directory]
- [start Traffic Editor from the parent directory]

```
rocker --nvidia --x11 --name traffic-editor --user \  
  --volume `pwd`/buildings:/rmf_demos_ws/buildings -- \  
  ghcr.io/open-rmf/rmf/rmf_demos:jazzy-rmf-latest \  
  bash
```

```
traffic-editor
```

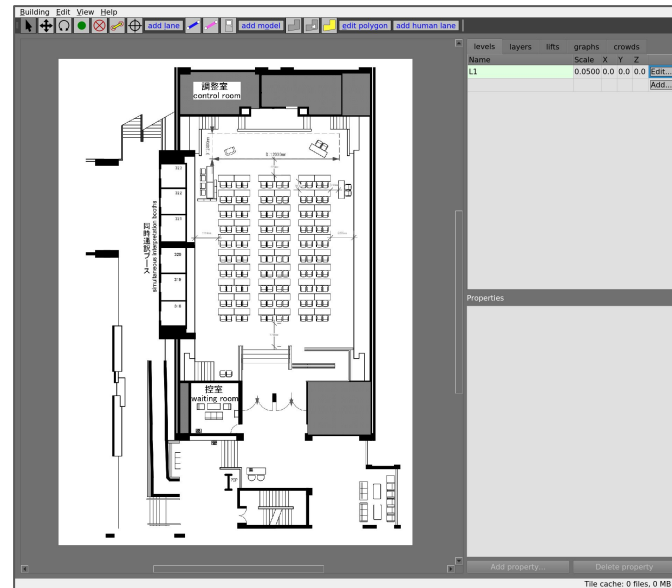
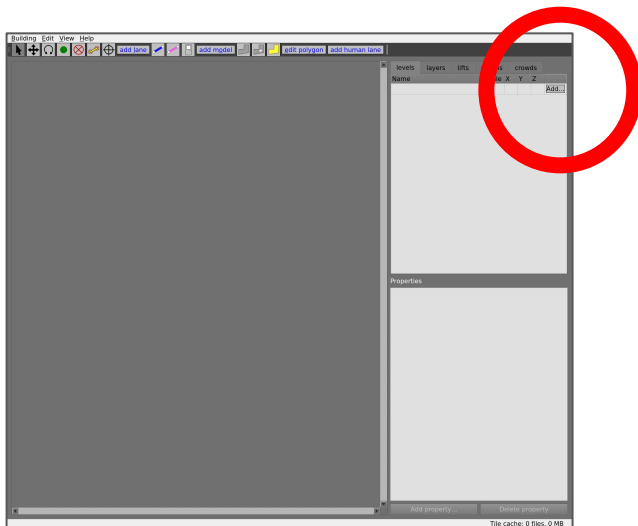
Step 2a(ii): create a traffic editor project

- Building → New... [name it something]
[save it in /rmf_demos_ws/buildings]
 - use "Reference-image coordinates"
- General Usage:
 - press [escape] at any time to return to the Select tool 
 - mouse scroll wheel for zoom
 - mouse middle-drag to pan
 - best with one hand on "real" mouse, other hand on keyboard (like CAD)



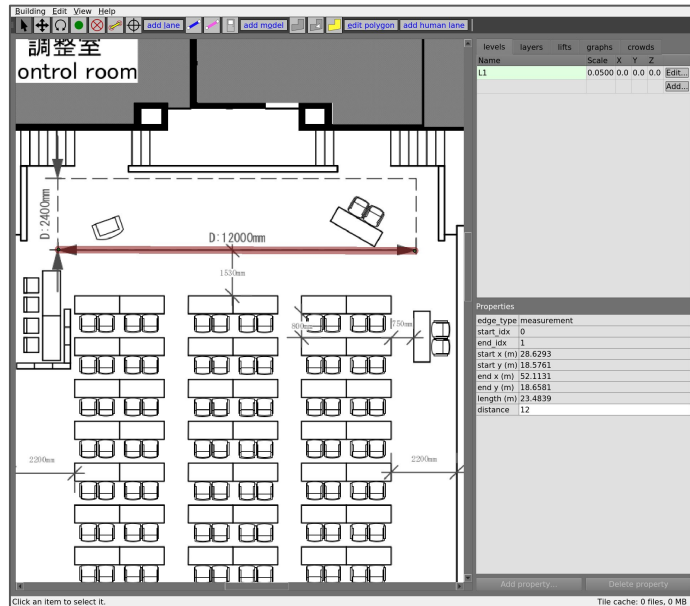
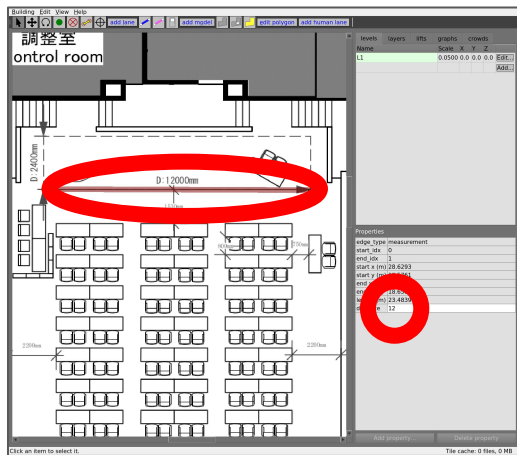
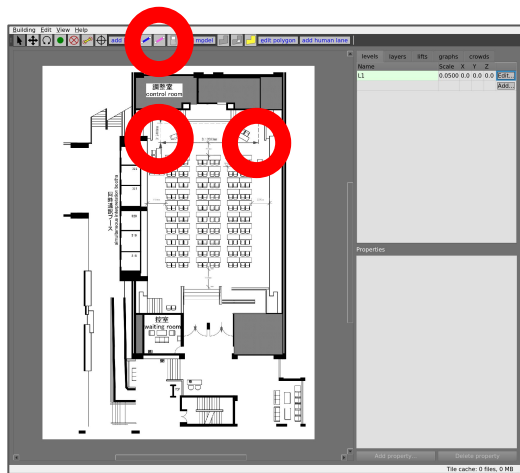
Step 2b: create level L1

- In the right "Levels" tab, click "Add..."
 - level name: L1
- drawing: click "Find..."
 - select the [PNG map image](#)
- click OK



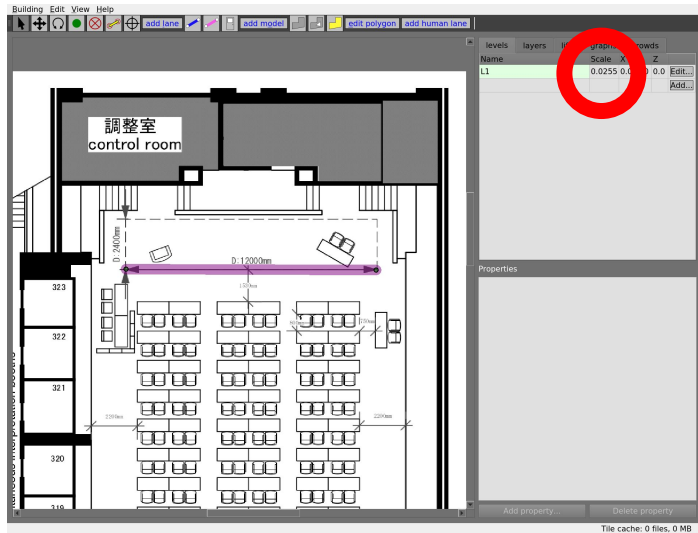
Step 2c: set level image scale

- Identify a known dimension in the drawing
- Click on the Measurement (pink line) tool
- Click on the left side of the dimension
- Click on the right side of the dimension
- Press [Escape] or click on the Select tool
- Click on the pink measurement line
- Type its dimension (in meters), press [Enter]



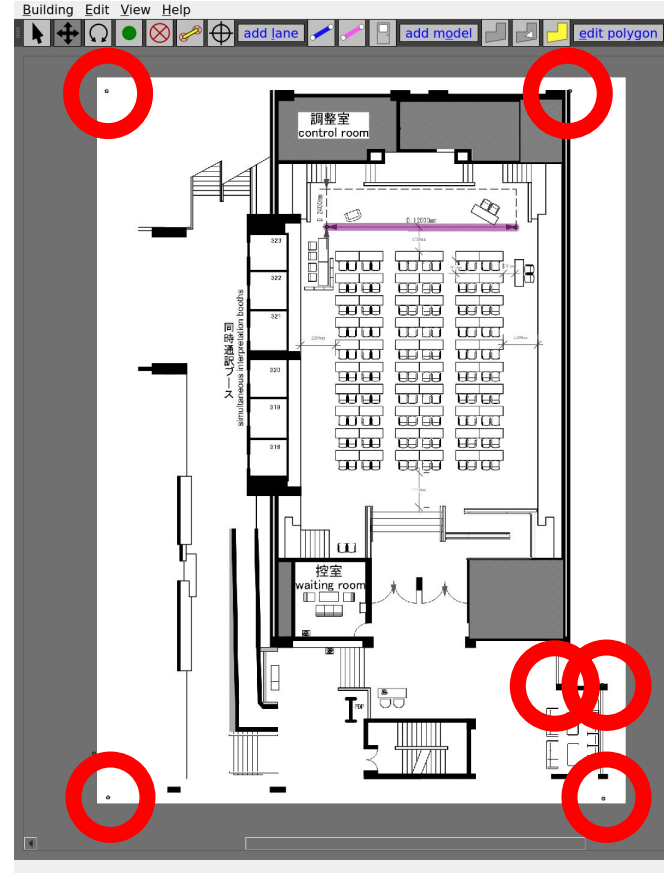
Step 2d: save and restart traffic-editor

- At time of writing 🤔 a restart is needed
 - Scale is calculated only at editor startup
- Building → Save... [type a filename]
- Building → Exit
- [start traffic editor again]
 - PRO TIP: traffic-editor NAME.building.yaml
- Scale is now 0.0255 meters/pixel 👍
 - If you add more measurements, the editor will average all of them
 - Don't have to be ultra precise
 - PRO TIP: use the longest measurement in the drawing to reduce error sensitivity



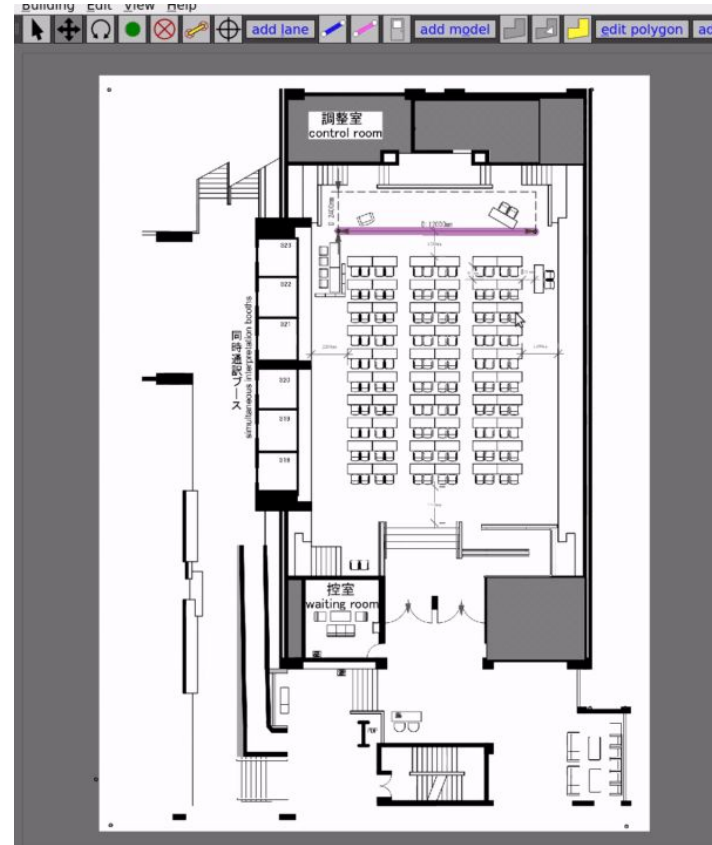
Step 2e (i): create floor polygon vertices

- We need to create the vertices that we will use for our floor polygon. Because our map is part of a larger building, the floor polygon will be "outside" some walls.
- Click on the Add Vertex tool 
 - left-click on corners we want for our floor
 - PRO TIP: zoom in/out so it's easier
 - PRO TIP: press [escape] when done 



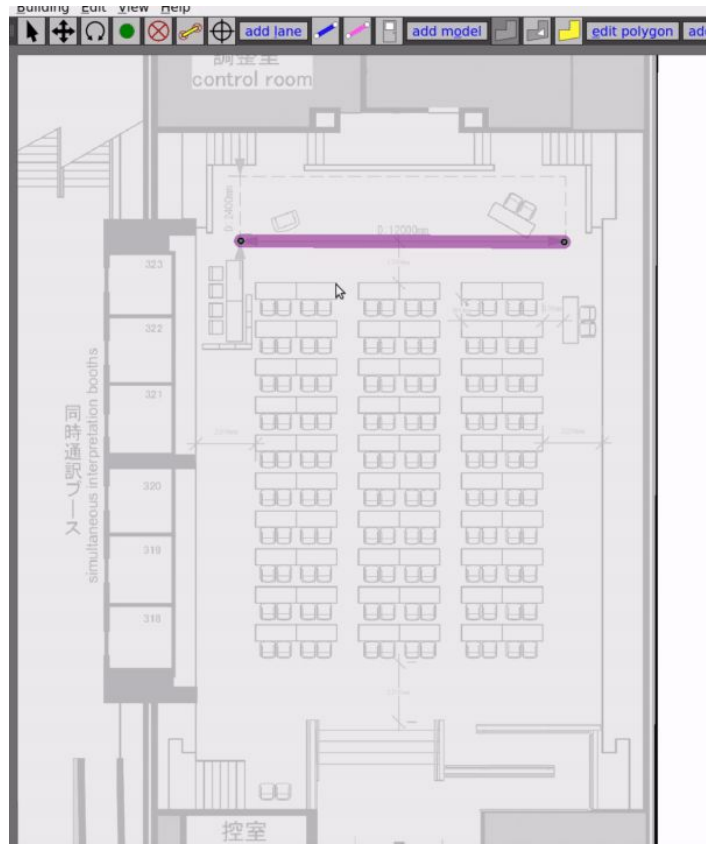
Step 2e (ii): create the floor polygon

- Click on the Add Polygon tool 
 - left-click the vertices you want
 - right click to finish and close the polygon
 - Press [escape] to return to Select tool 
- Save and celebrate 😊
 - PRO TIP: Ctrl+S



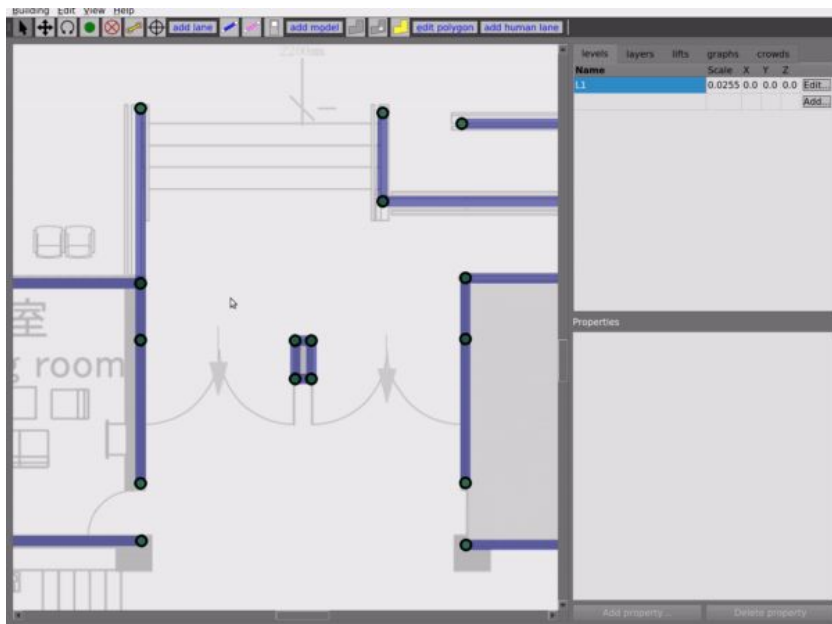
Step 2f: create some walls

- Click on the Add Wall tool 
 - PRO TIP: press [w]
- The first left-click starts a wall
- each additional click adds a wall segment
 - PRO TIP: hold shift to constrain to horizontal/vertical
- Right-click to finish a wall chain
- Continue adding wall chains until you're bored
- Press [escape] to return to Select tool
- Save and celebrate 😊

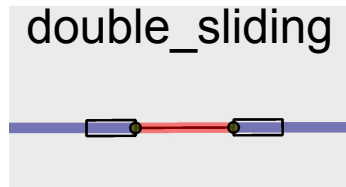
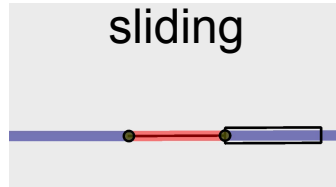
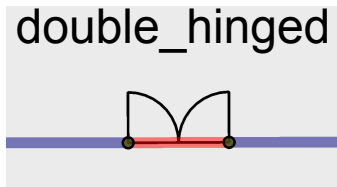
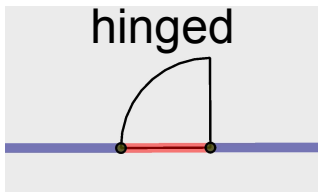


Step 2g (i): create some doors

- Create walls with a "gap" for a door
- Click on the Add Door tool 
- PRO TIP: press [d]
- First left-click starts a door segment
- Second left-click ends a door segment
- Continue adding doors until you're bored
- Press [escape] to return to Select tool
- Save and celebrate 😊

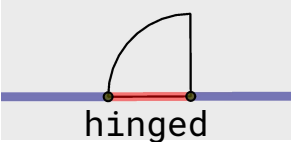
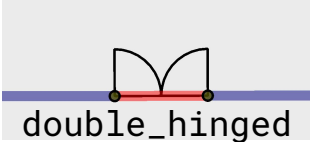
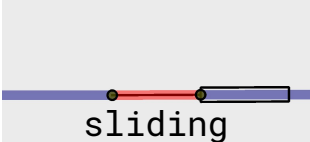
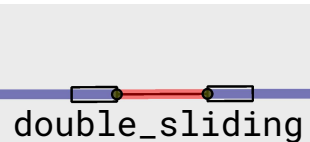
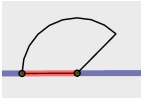
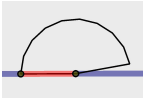
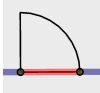
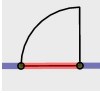
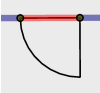
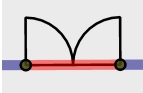
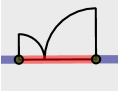
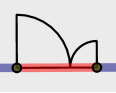


Use the "type" parameter to describe the door



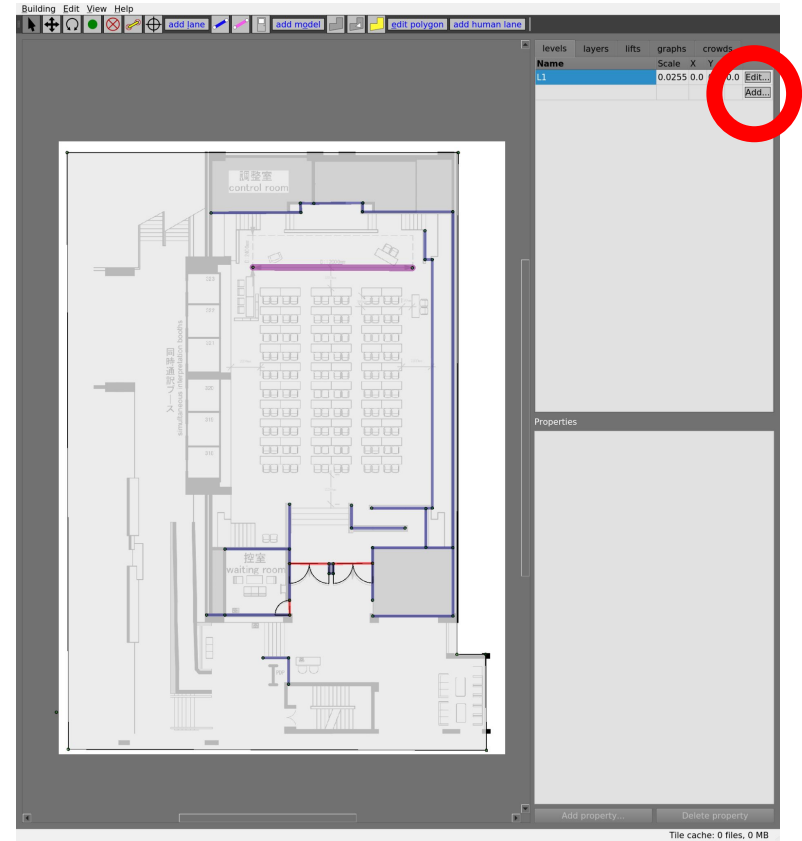
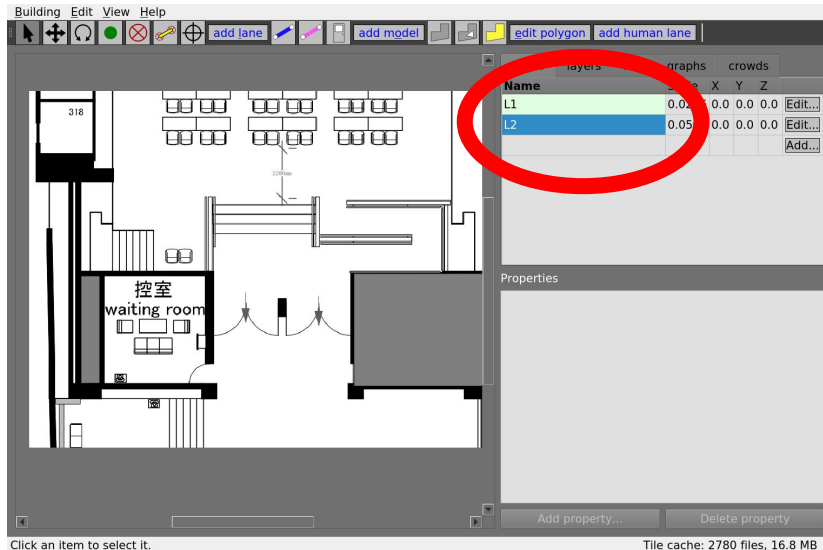
Step 2g (ii): configure door parameters

- Doors are surprisingly complicated!
- Use the property editor to configure them

type	 hinged	 double_hinged	 sliding	 double_sliding
motion_degrees	90 	135 	170 	
motion_axis	start 	end 		
motion_direction	1 	-1 		
right_left_ratio	1 	2 	0.5 	

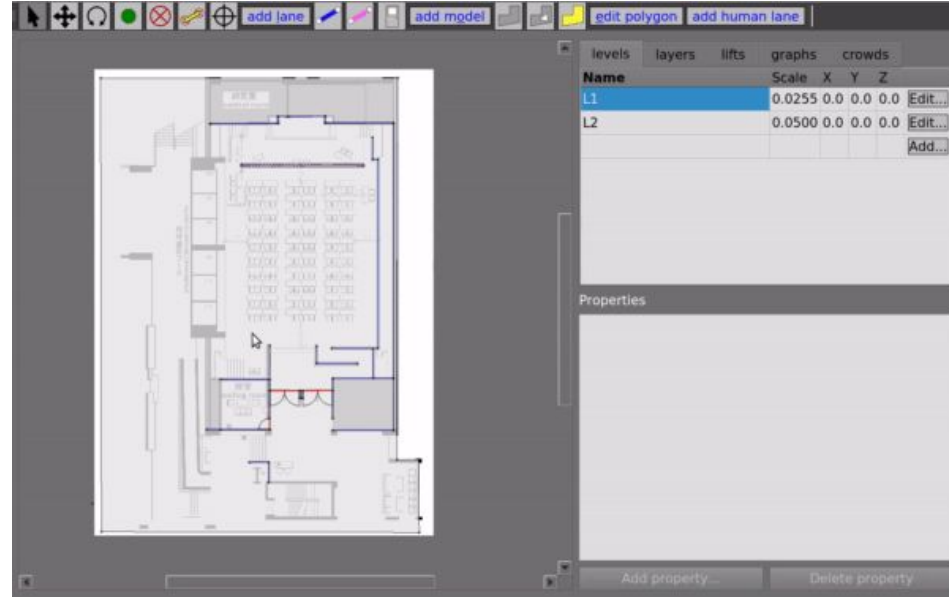
Step 2h: add level L2

- Click "Add..." on the Levels tab
 - name: L2
 - elevation: 8 [meters]
 - drawing: [[find PNG](#)]
- Save 😊
- Now you can click the level row to toggle



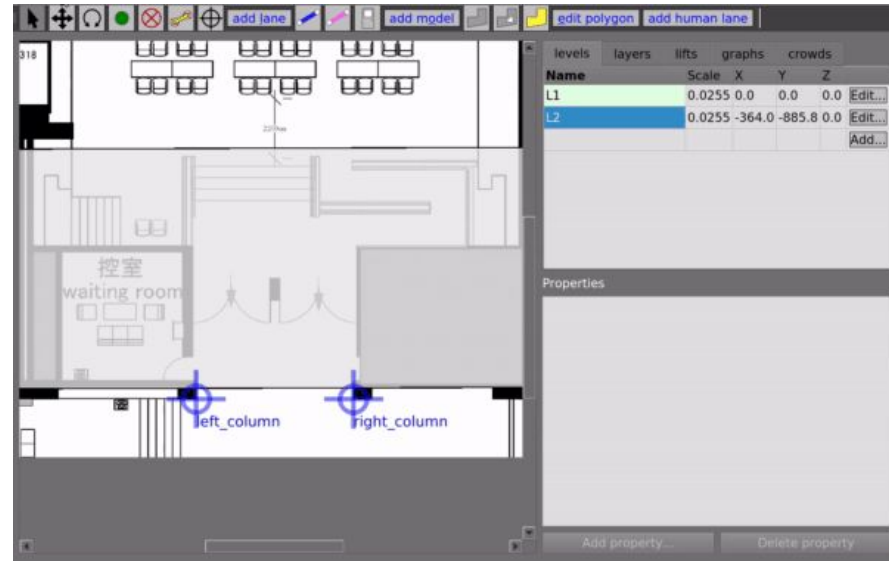
Step 2h (i): add level-alignment fiducials

- We must align level maps vertically
- Level maps are often different resolutions, scales, and sizes
- To deal with this, we add Fiducials
 - things we can find in multiple levels that are aligned vertically
 - example: structural columns
- Click the "Add Fiducial" tool 
- Click on ≥ 2 places in both maps
- Name each vertical-pair the same
- Save 😊
- Restart traffic-editor 😬



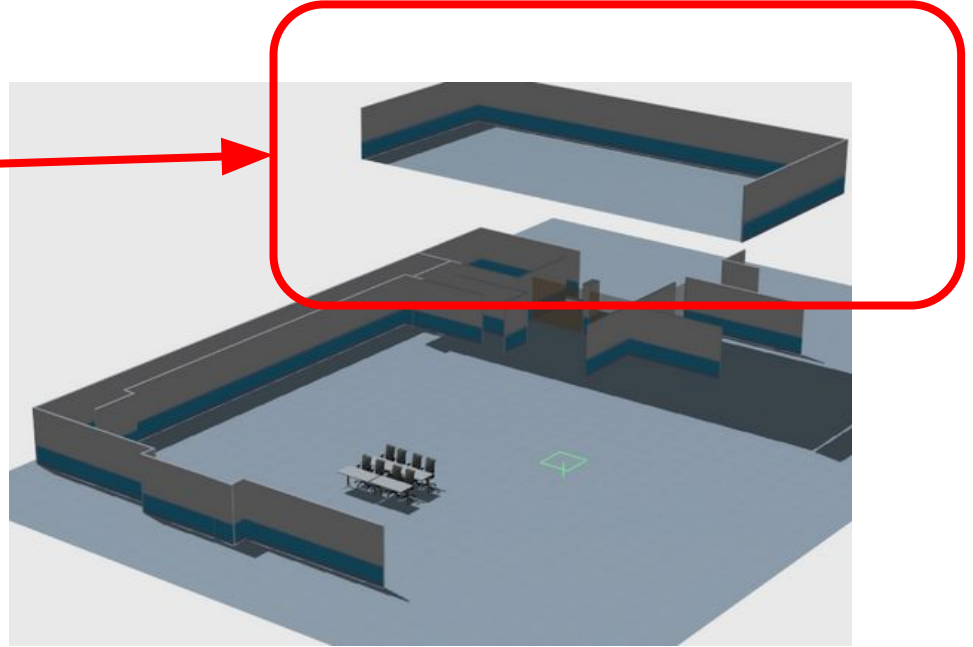
Step 2h (ii): ensure level alignment works

- After restarting traffic-editor 🤔
zoom into an area and toggle levels
- If fiducial alignment worked, the main working area should "stay in the same place" when you switch levels
- If this does not happen, ensure your vertically-aligned fiducials have EXACTLY the same name!
 - restart when changing them 🤔



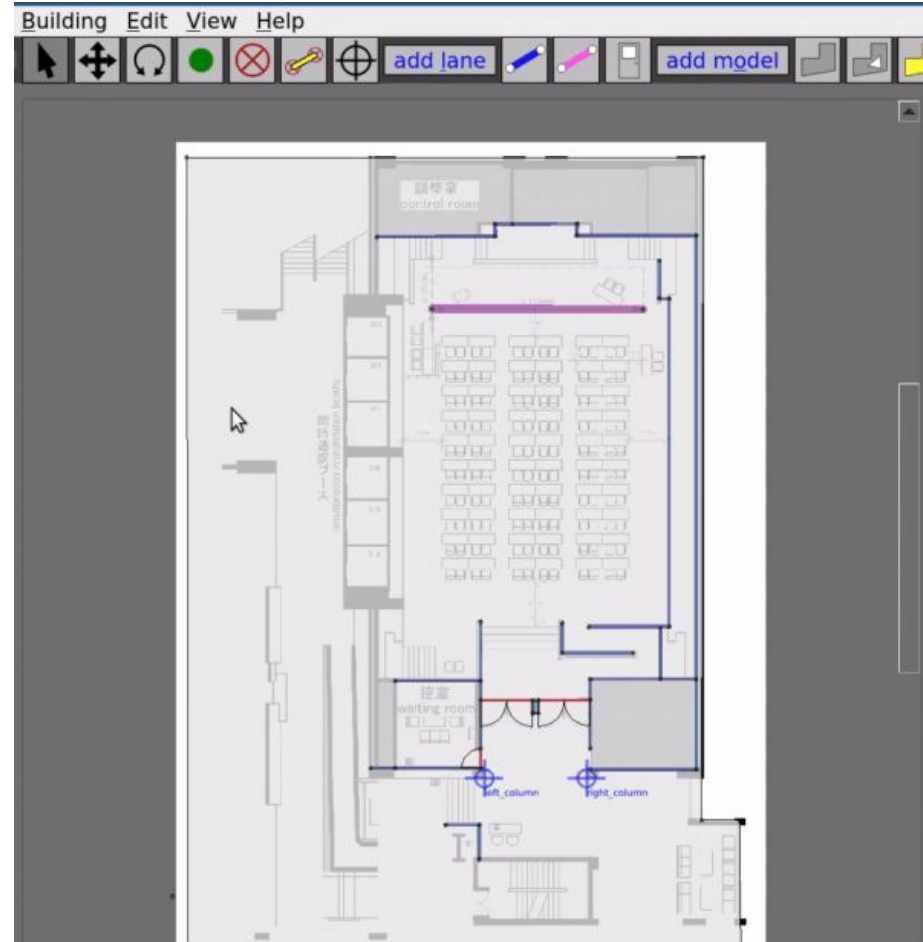
Step 2h (iii): create 2nd floor

- Create floor polygon vertices
- Create floor polygon
- Create walls



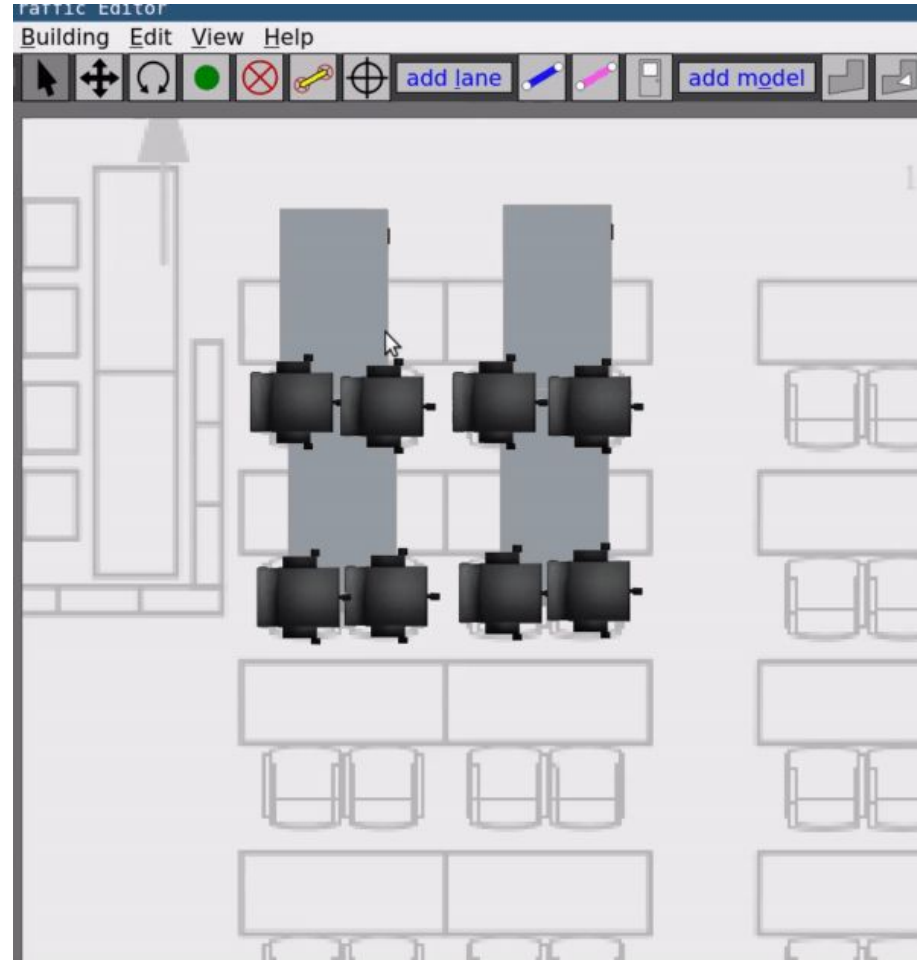
Step 2i: add some furniture

- Let's add some tables and chairs
- Click the Add Model tool **add_model**
- Select a model
 - OpenRobotics/AdjTable
 - OpenRobotics/OfficeChairGrey
- Left-click to place in the world
 - rotation will be fixed later
 - can "fine-tune" position later



Step 2j: rotate and move furniture

- Click the Rotate tool 
- Left-drag on a model to rotate it
 - PRO TIP: hold [shift] key for 45-degree presets
- Continue until all models are facing the correct cardinal direction
- Click on the Move tool 
- Left-drag to move



Step 2k: generate a Gazebo world!

- Now we can generate a Gazebo world
- Typically this is done in a WORLDNAME_maps package CMake
- We can also run the generator manually:

cd buildings/

```
ros2 run rmf_building_map_tools building_map_generator \  
gazebo icc_kyoto.building.yaml icc_kyoto.world ./icc_kyoto_world
```

- `ros2 run rmf_building_map_tools building_map_generator` = run program
- `gazebo` = thing to generate (a Gazebo world)
- `icc_kyoto.building.yaml` = our Traffic Editor file name
- `icc_kyoto.world` = the Gazebo world file name to generate
- `./icc_kyoto_world` = directory to hold the generated wall and floor meshes

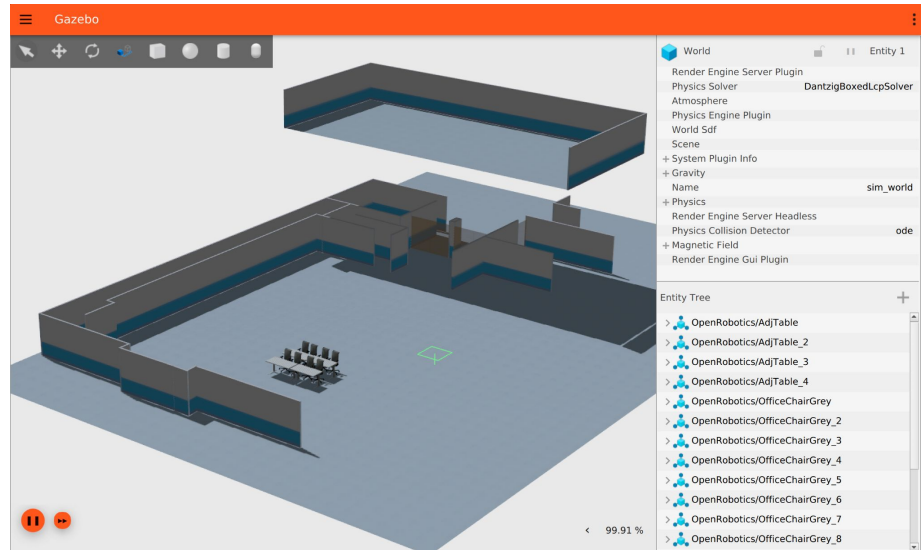
Replace “icc_kyoto” with the name of YOUR building!!!

Step 2I: view the Gazebo world!

- Typically this is done in CMake, but we can run the tools manually

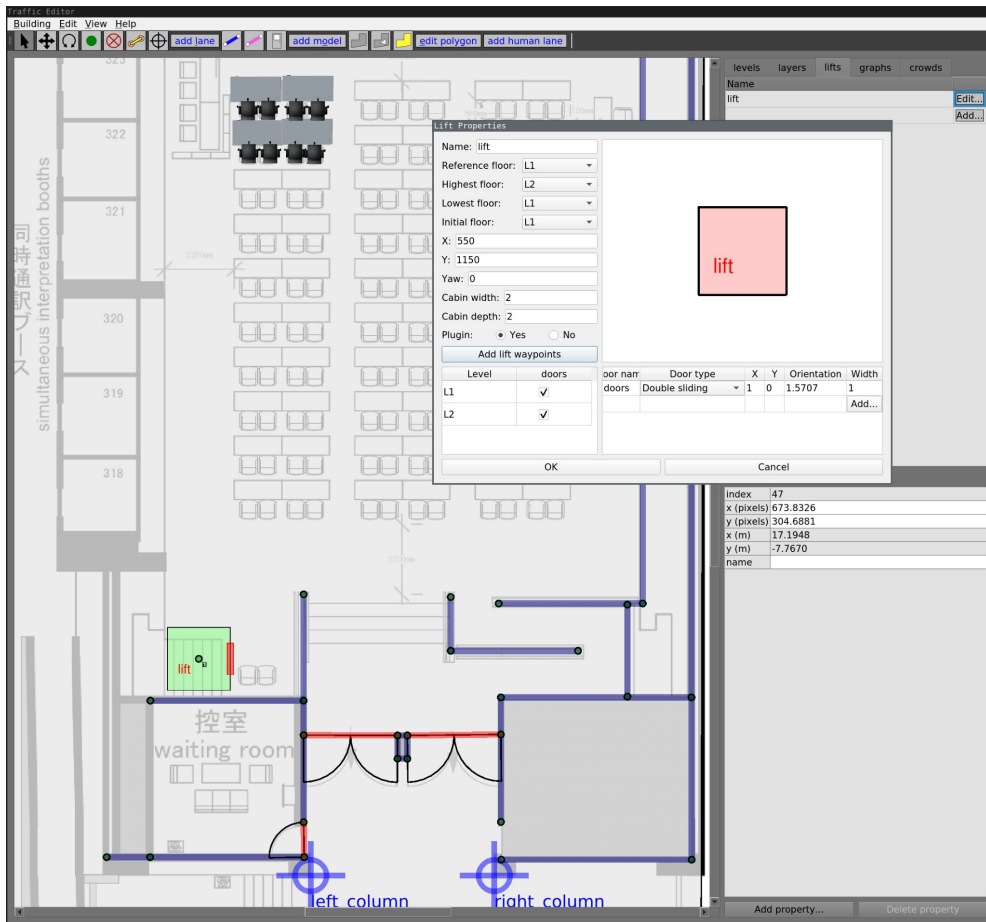
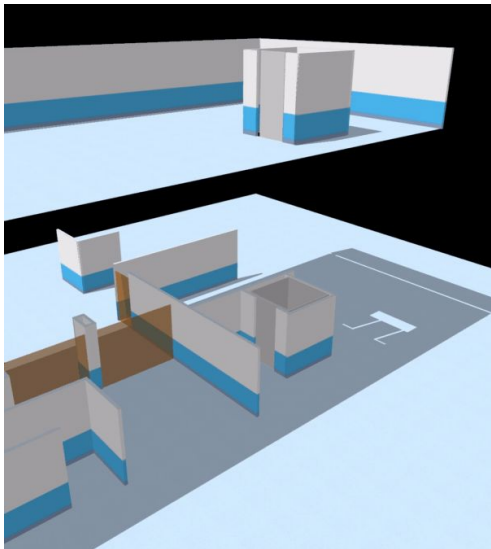
```
ros2 run rmf_building_map_tools building_map_model_downloader \  
icc_kyoto.building.yaml -e ./models
```

```
export GZ_SIM_RESOURCE_PATH=`pwd`/icc_kyoto_world: `pwd`/models  
gz sim -r -v 4 icc_kyoto.world
```



Step 2m: add an elevator

- Make sure level L1 is active
- Click the "Lifts" tab (upper-right)
- Click "Add..."
 - fill out as shown
 - click "Add..." in doors table
 - X, Y takes iteration 😬

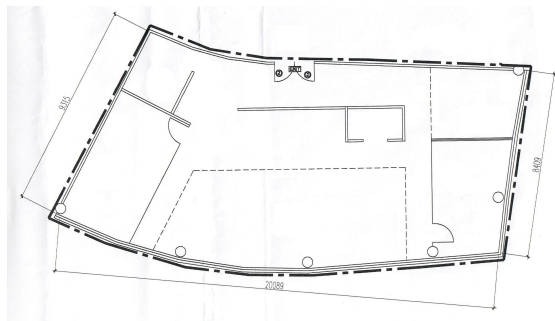


Start using Open-RMF in three easy steps

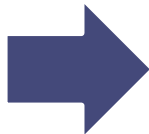
1. Find or create a building map 
2. Create a simulation model 
3. **Make a robot traffic map**

Step 1 may require human interaction.

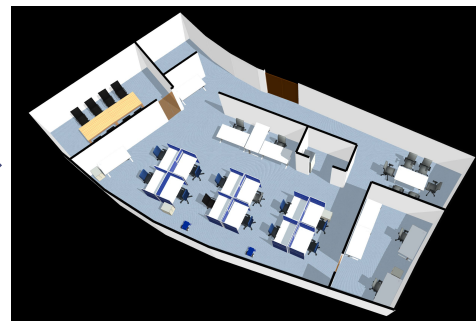
Traffic Editor helps you with steps 2 and 3.



Building floorplan

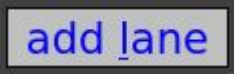


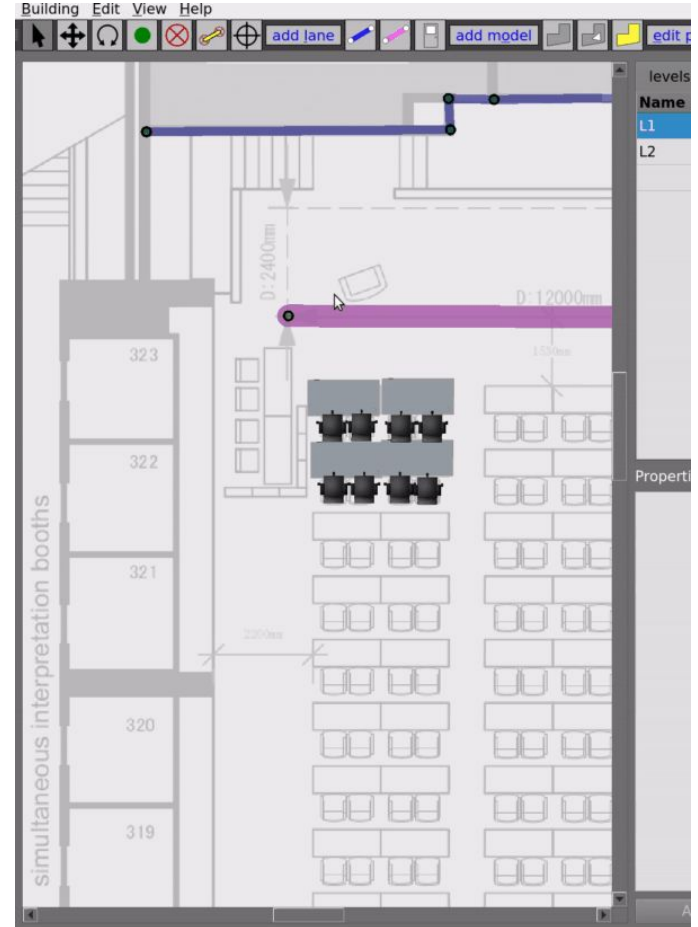
Annotated in *traffic_editor*



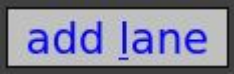
Physics-based Gazebo
simulation world

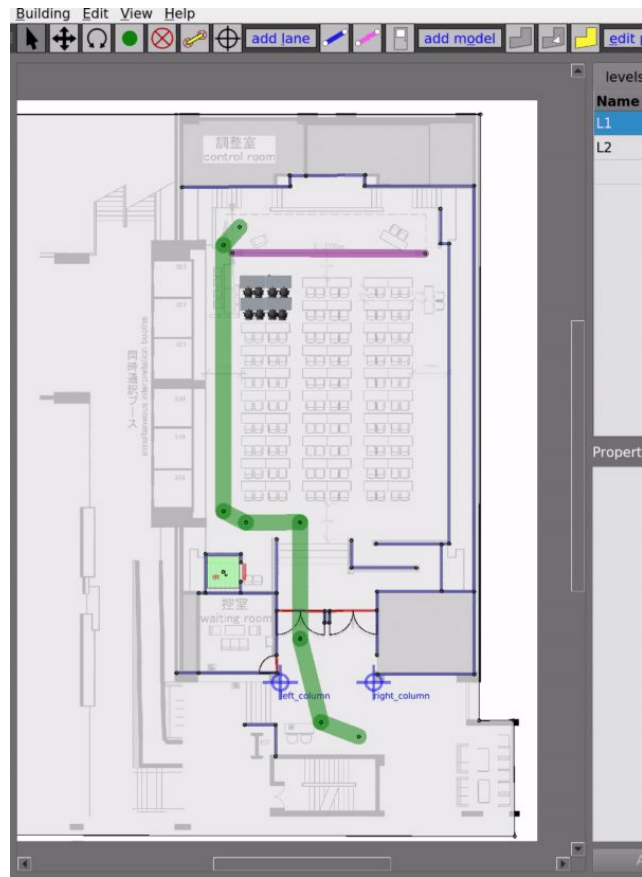
Step 3a: create some robot traffic lanes

- Click on the Add Lane tool 
 - PRO TIP: press [l] (lower-case L)
- The first left-click starts a lane segment
- each additional click adds a lane segment
 - PRO TIP: hold shift to constrain to horizontal/vertical
- Right-click to finish a lane chain
- Continue adding lanes until you're bored
- Press [escape] to return to Select tool
- Save and celebrate 😊



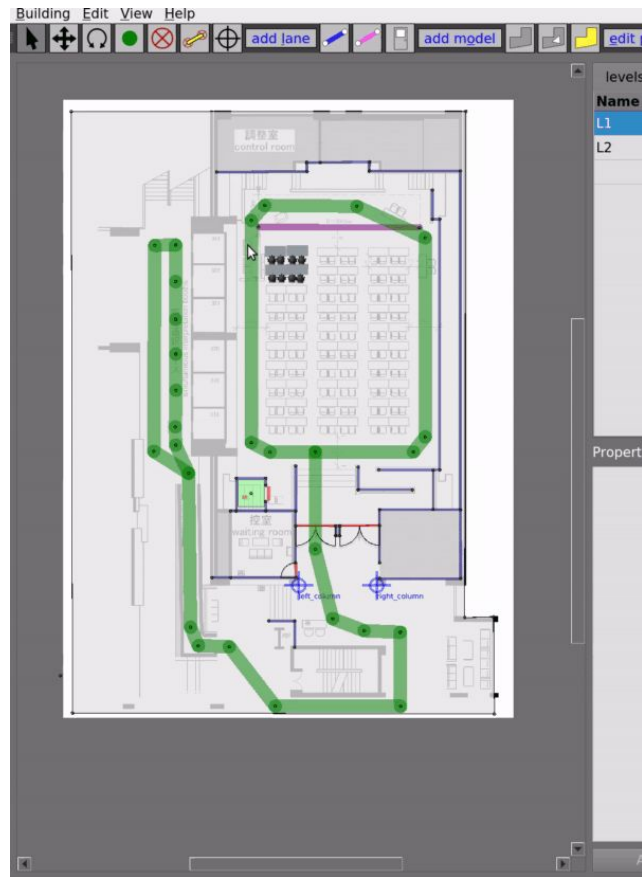
Step 3b: branch lane segments

- Click on the Add Lane tool 
 - PRO TIP: press [l] (lower-case L)
- Zoom in on a vertex you'd like to branch
- Left-click **directly** on the vertex
 - PRO TIP: a slight "mis-click" can result in a disconnected lane graph and sadness
- Continue left-clicking to extend the branch
- Right-click to finish a lane chain
- Continue adding lanes until you're bored
- Press [escape] to return to Select tool
- Save and celebrate 😊



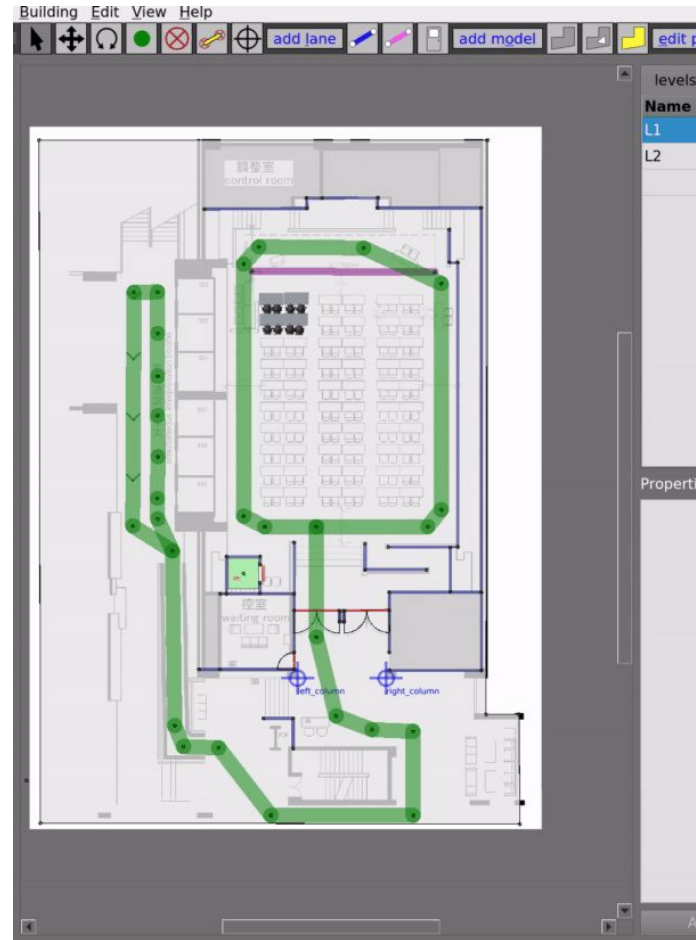
Step 3c: one-way lane segments

- Make sure you're in the Select tool 
 - PRO TIP: press [escape]
- Zoom in on a lane segment
- Left-click to select (it will turn red)
- Option 1:
 - change the bidirectional parameter to **false**
- Option 2: PRO TIP
 - press [b]
- Continue left-clicking to extend the branch
- Right-click to finish a lane chain
- Continue adding lanes until you're bored
- Press [escape] to return to Select tool
- Save and celebrate 😊



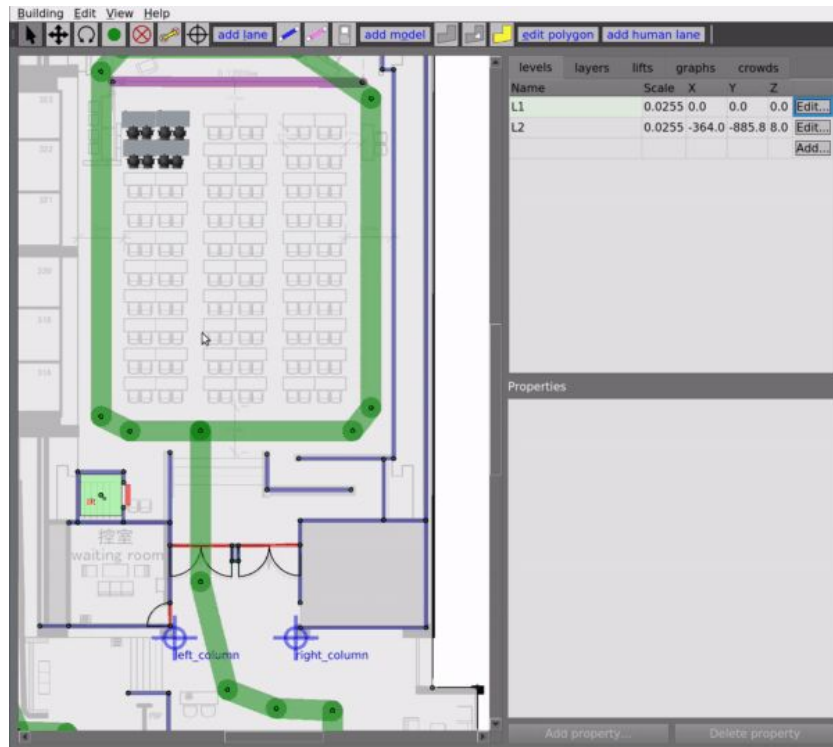
Step 3d: editing lane segments

- Make sure you're in the Select tool 
 - PRO TIP: press [escape]
- Zoom in on a lane segment
- Left-click to select (it will turn red)
- Delete it with the **[delete]** key
 - Edit→Undo the deletion (Ctrl+Z)
- Click on the Move Tool 
 - PRO TIP: press [m]
- Left-drag a vertex to move it
 - all attached segments should drag with it
- Press [escape] to return to Select tool
- Save and celebrate 😊



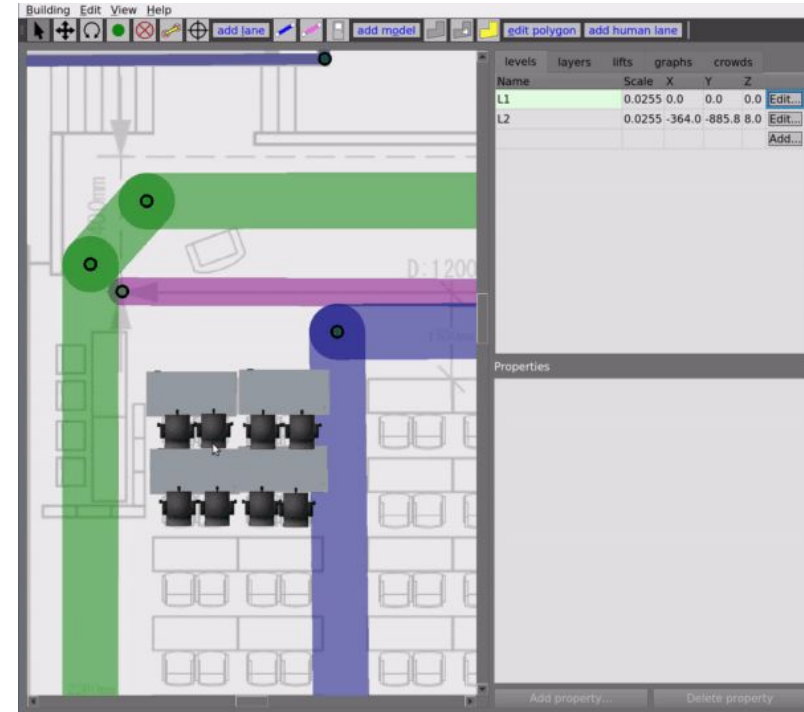
Step 3e: adding another lane graph

- Select the "Graphs" tab at upper-right
- Choose a different graph, like "Graph 1"
- Add lanes as usual
- Graphs have unique colors
- PRO TIP: toggle between graphs by pressing the number keys
 - [0] to edit Graph 0
 - [1] to edit Graph 1
 - [2] to edit Graph 2, etc.



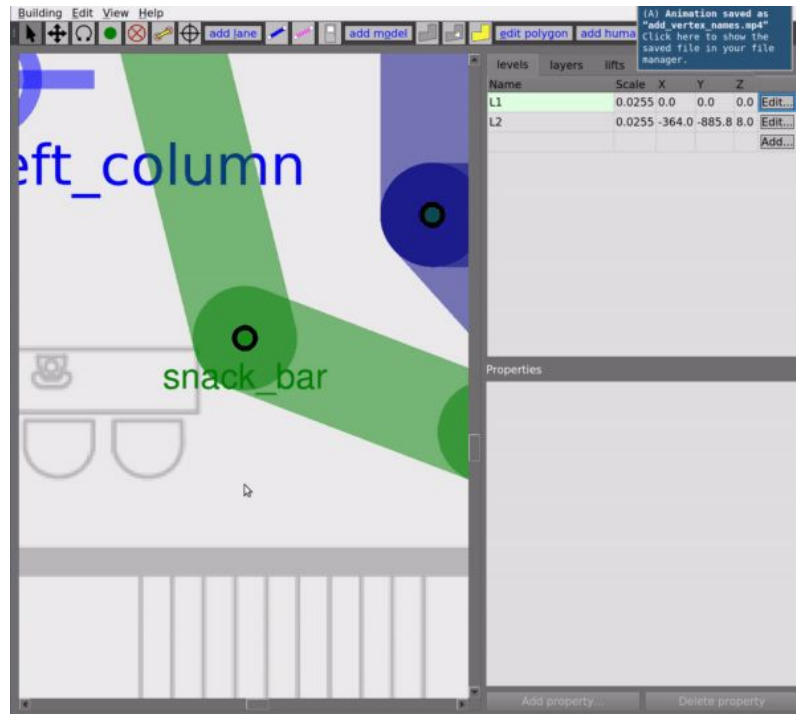
Step 3f: vertex names

- Press [escape] to ensure Select mode 
- Left-click on any vertex (green circle)
- Enter something in the "name" property



Step 3g: vertex properties

- Press [escape] to ensure Select mode 
- Left-click on any vertex (green circle)
- If a property is visible, you can edit it
 - be sure to press [enter] afterwards
- Otherwise, click "Add property..." and add the desired property to the vertex



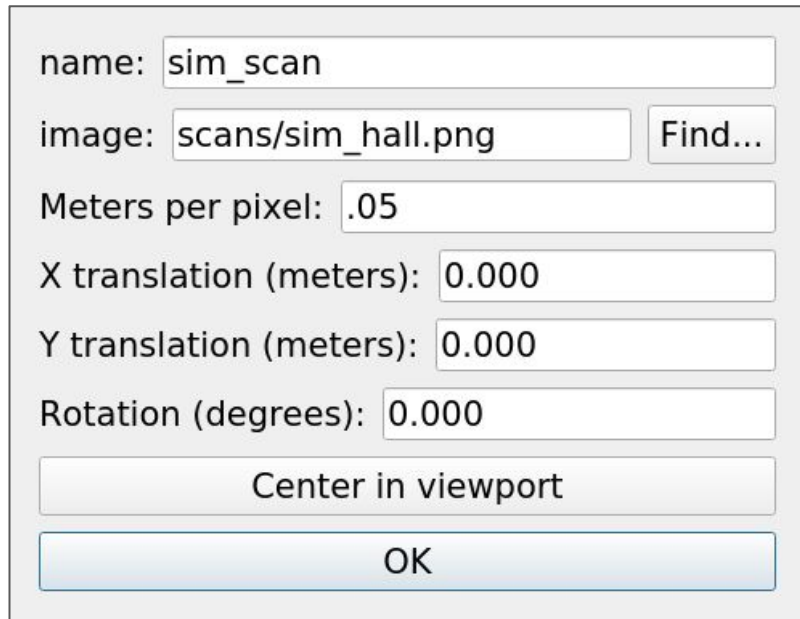
Step 3g: vertex properties

- Press [escape] to ensure Select mode 
- Left-click on any vertex (green circle)
- If a property is visible, you can edit it
 - be sure to press [enter] afterwards
- Otherwise, click "Add property..." and add the desired property to the vertex

```
is_parking_spot
is_charger
dock_name
is_cleaning_zone
dropoff_ingestor
pickup_dispenser
spawn_robot_type
spawn_robot_name
is_holding_point
is_passthrough_point
human_goal_set_name
mutex
merge_radius
```

Step 3h: add a robot LIDAR map layer

- Select the Layers tab in the upper-right
- Click on the "Add..." button
- Find the map layer file
- Enter starting guess for the resolution
 - often, LIDAR maps are 0.05 m/pixel



name:

image:

Meters per pixel:

X translation (meters):

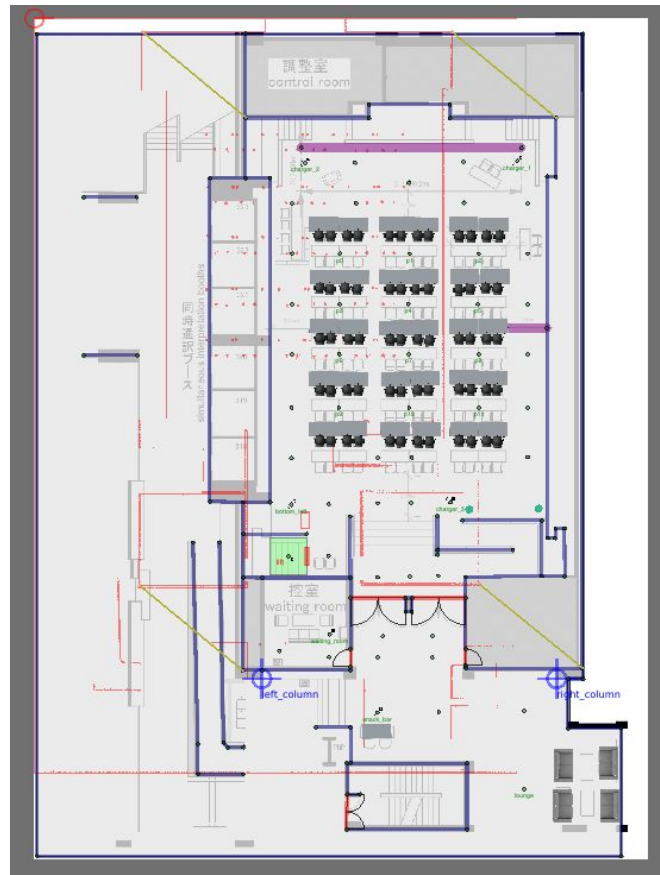
Y translation (meters):

Rotation (degrees):

Download image from [here](#).

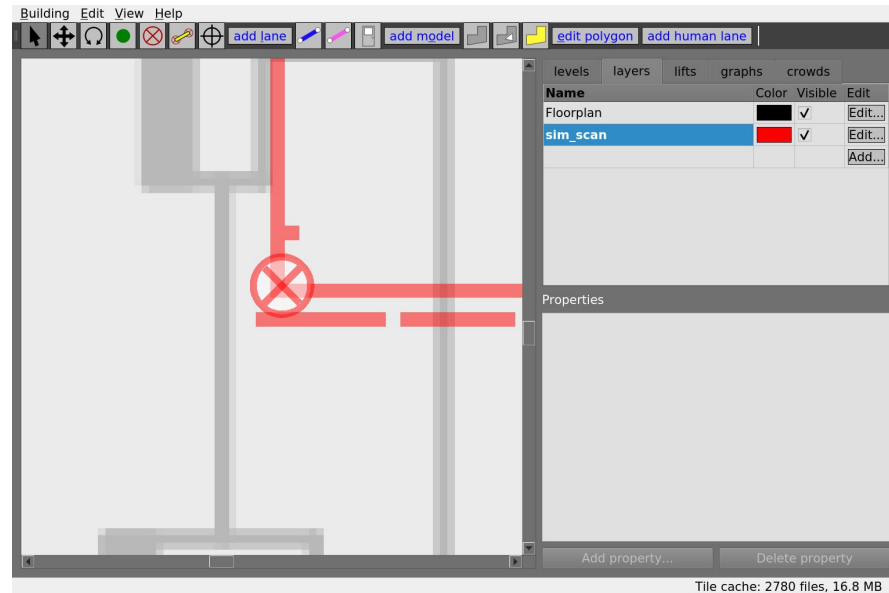
Step 3i: LIDAR map layer alignment overview

- The robot coordinate system typically has (0,0) where the robot was physically located when powered up
- This is almost certainly not the upper-left corner of the map!
- Our task is to align the two maps: define a rotation, translation, and scaling which makes them "line up"
- Option 1:
 - manually iterate parameters
- Option 2:
 - identify common features in the LIDAR map and define "constraints"
 - use a solver to minimize the constraints



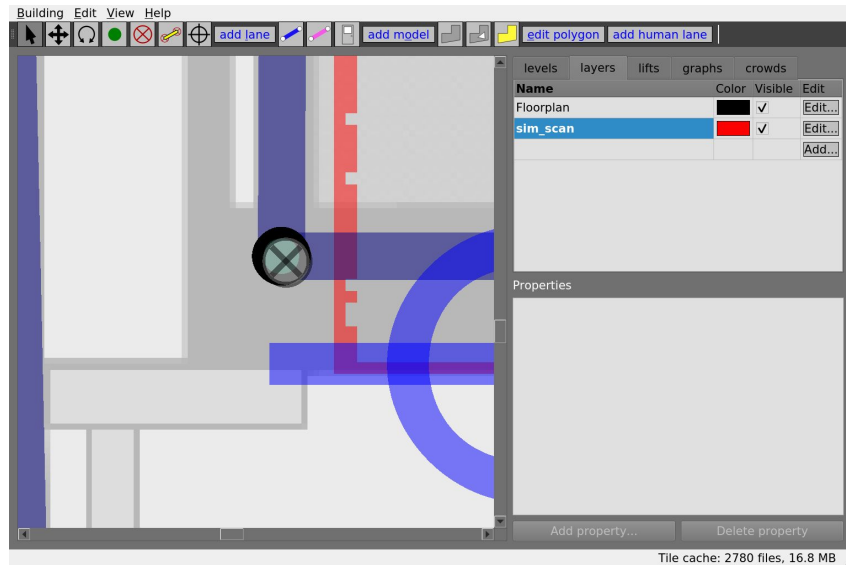
Step 3j: annotate features in robot map

- Zoom in on the feature in the robot map
- Select the robot map layer (red) in the layer table at right
- Click on the "Add Feature" tool 
- Click on the location of the feature
- Ensure that the feature marker is the same color as the map!
- Repeat for at least one other feature (ideally more)



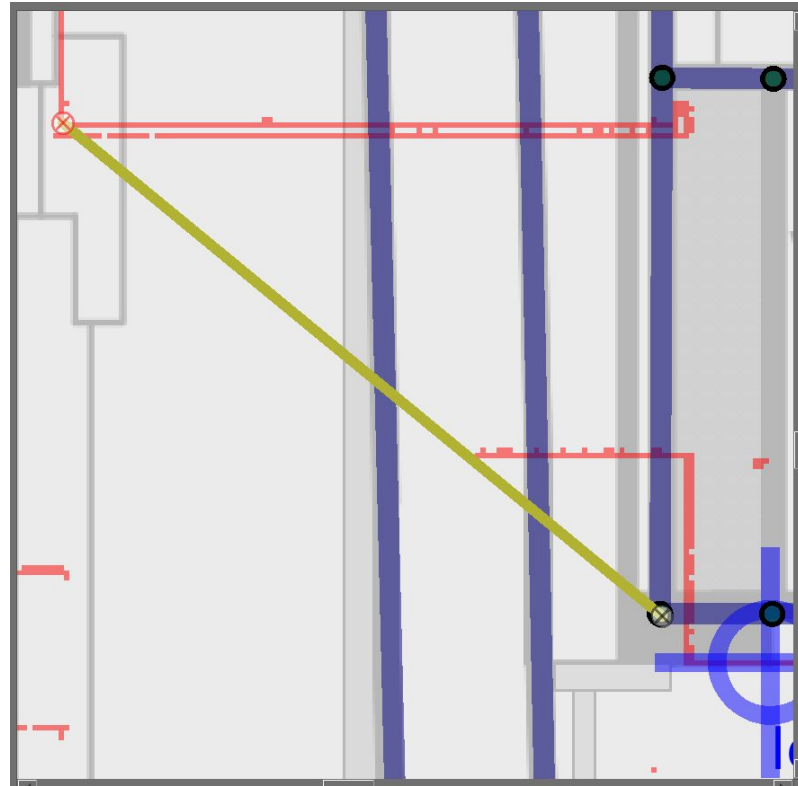
Step 3k: annotate features in floorplan

- Zoom in on the feature in the floorplan
- Select the Floorplan layer (black) in the layer table at right
- Click on the "Add Feature" tool 
- Click on the location of the feature
- Ensure that the feature marker is the same color as the floorplan (black)!
- Repeat for at least one other feature (ideally more)



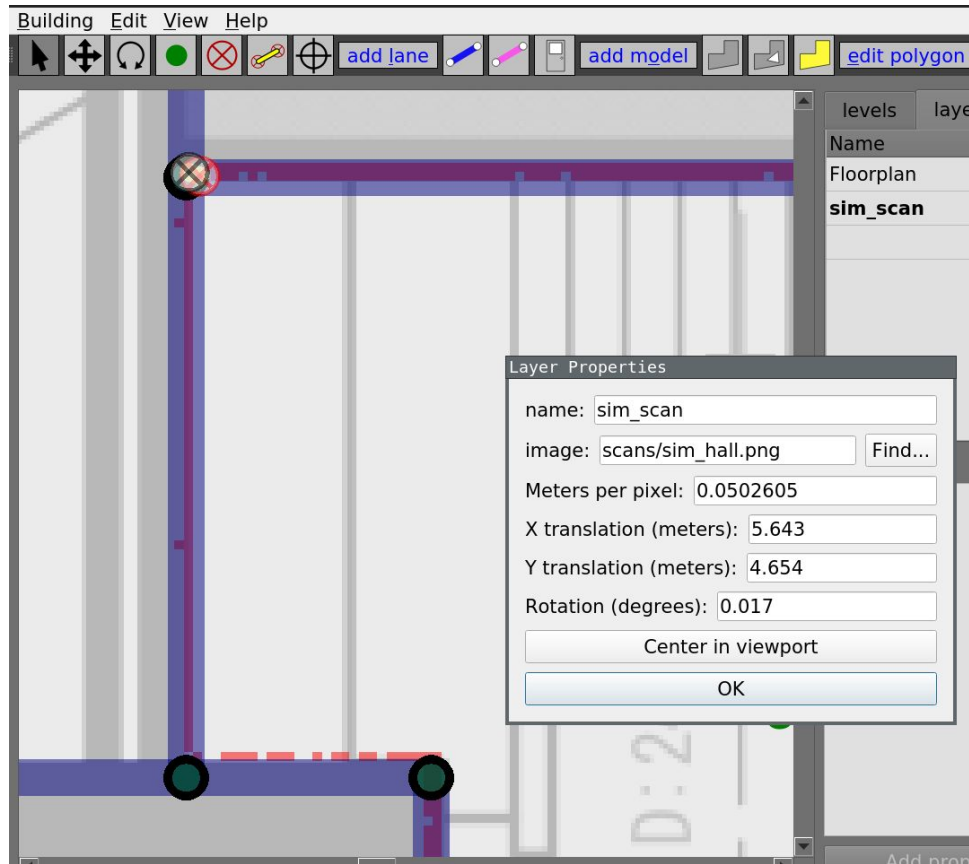
Step 3I: create constraints

- Zoom so that you can see the feature in both the floorplan and the robot map
- Use the "Add Constraint" tool 
- Click on the feature in the robot map
- Click on the feature in the floor plan
- A constraint will be created
 - this is a "spring" or "rubber band" that the solver will try to minimize
- Repeat for all other pairs of features



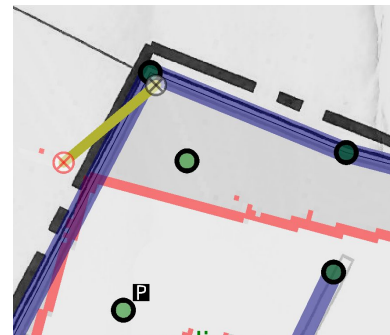
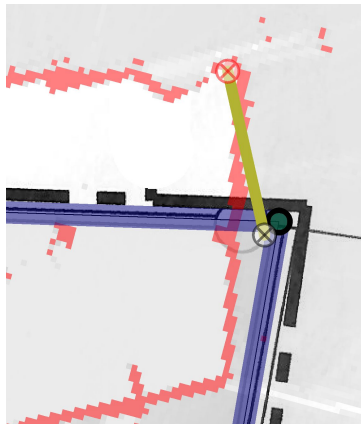
Step 3m: use constraint minimization algorithm

- Edit → Optimize Layer Transforms
 - PRO TIP: Ctrl+T
- Sometimes it helps to run it more than once
- Sometimes you'll need to add more constraints, or move existing constraints, to end up with a nice solution
- Typical solution is like this, with meters/pixel within a few percent

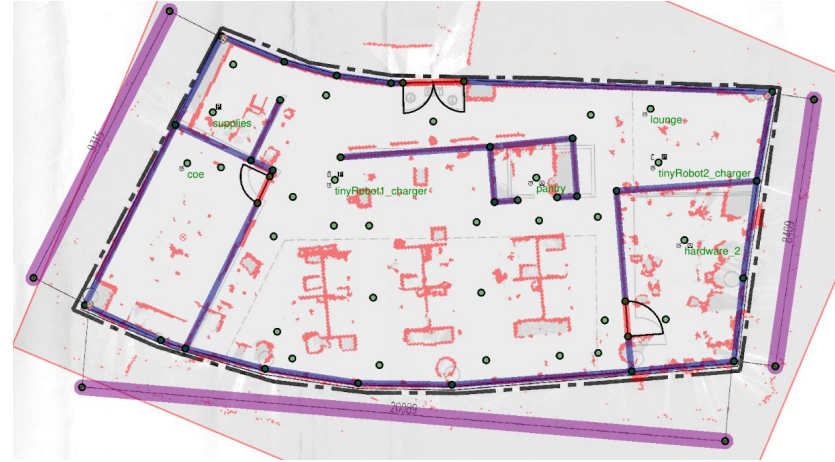
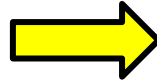
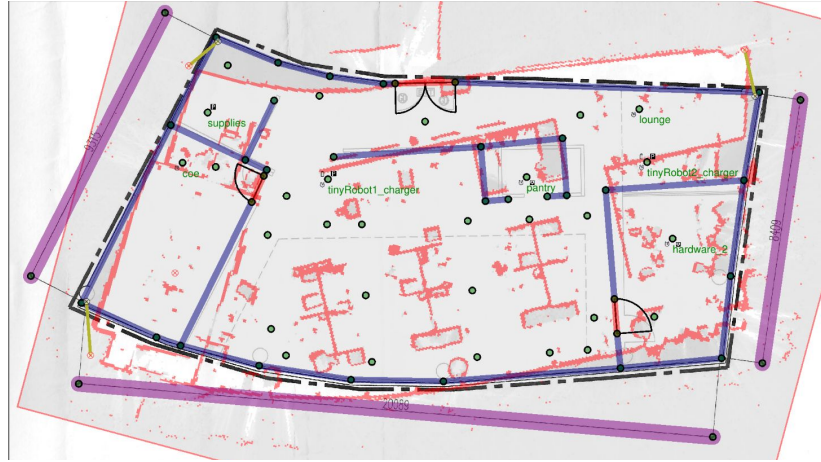


Step 3m (ii): real-data examples

- Fully-automatic alignment is hard
 - As-built often differs from floorplan, especially non-load-bearing walls
 - Cabinets, pillars, chairs, etc., can occlude walls
- Humans can easily spot these issues and ignore them
 - Computers cannot
- "Semi-Automatic" alignment workflow:
 - Roughly align the robot map using manual transform parameters
 - Click "features" that you can see in both robot map and floorplan
 - corners of load-bearing walls
 - pillars
 - Click "constraints" between corresponding features
 - Select Edit->"Optimize Layer Transforms" (Ctrl+T)
 - Numeric solver tries to find a transformation that minimizes the constraint lengths



Step 3m (iii): real-world map alignment



name:

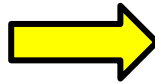
image:

Meters per pixel:

X translation (meters):

Y translation (meters):

Rotation (degrees):



name:

image:

Meters per pixel:

X translation (meters):

Y translation (meters):

Rotation (degrees):



Layer Properties

name: turtlebot

image: tb2.png Find...

Meters per pixel: 0.0511115

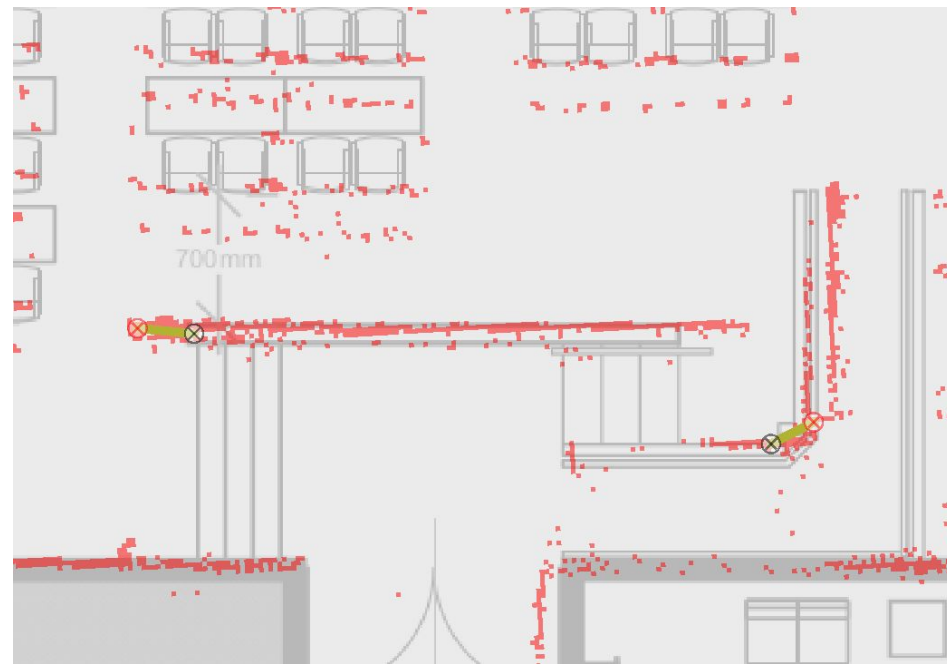
X translation (meters): 23.388

Y translation (meters): 26.862

Rotation (degrees): 183.443

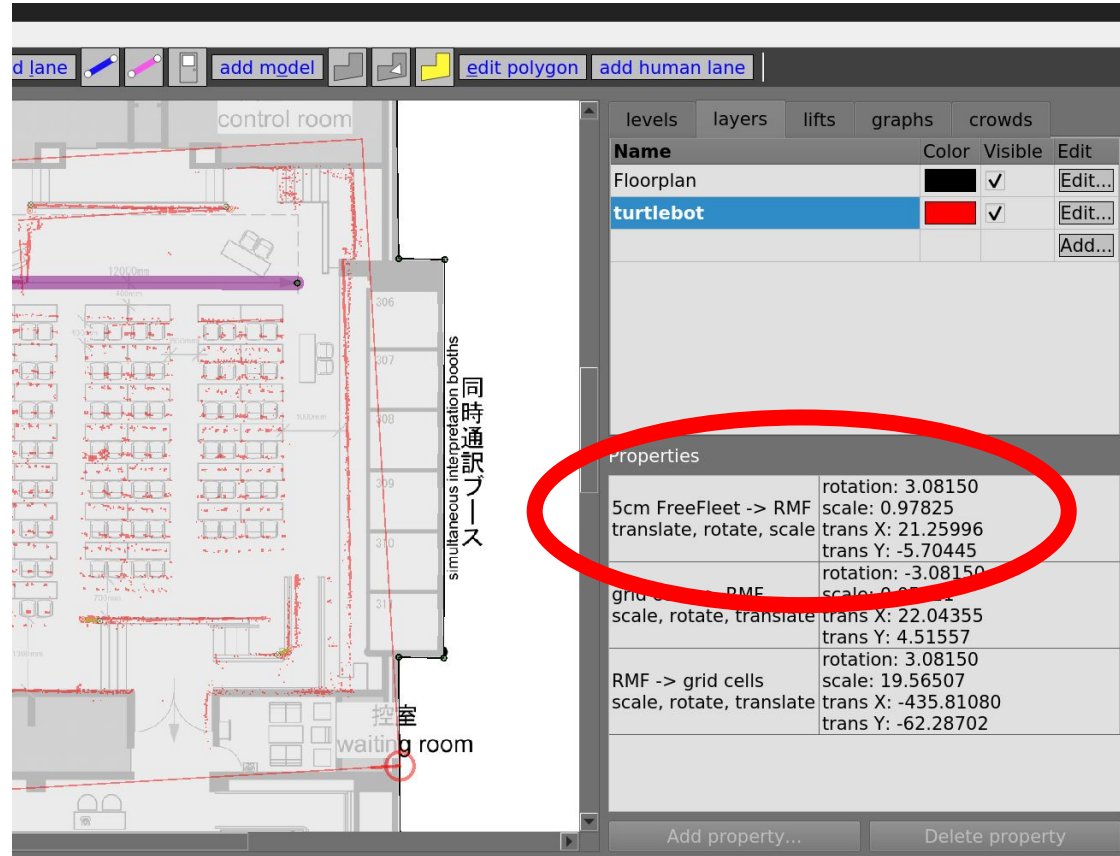
Center in viewport

OK



Step 3n: using the aligned map

- Somehow, need to use the transform rendered in Traffic Editor with your fleet adapter
- for FreeFleet, the parameters are computed
- Ensure to compensate for your offset! Try ensuring your map.yaml file has (0, 0) offset

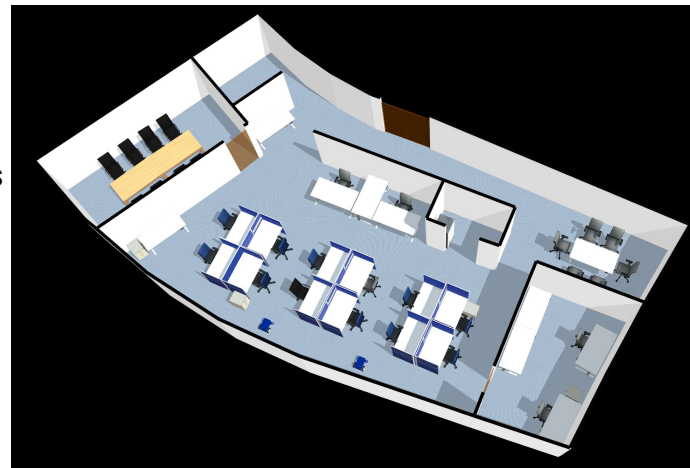


Building Map Tools



Annotated map in *traffic_editor*

building_map_tools



Physics-based simulation world with
3d assets, robots, doors, plugins

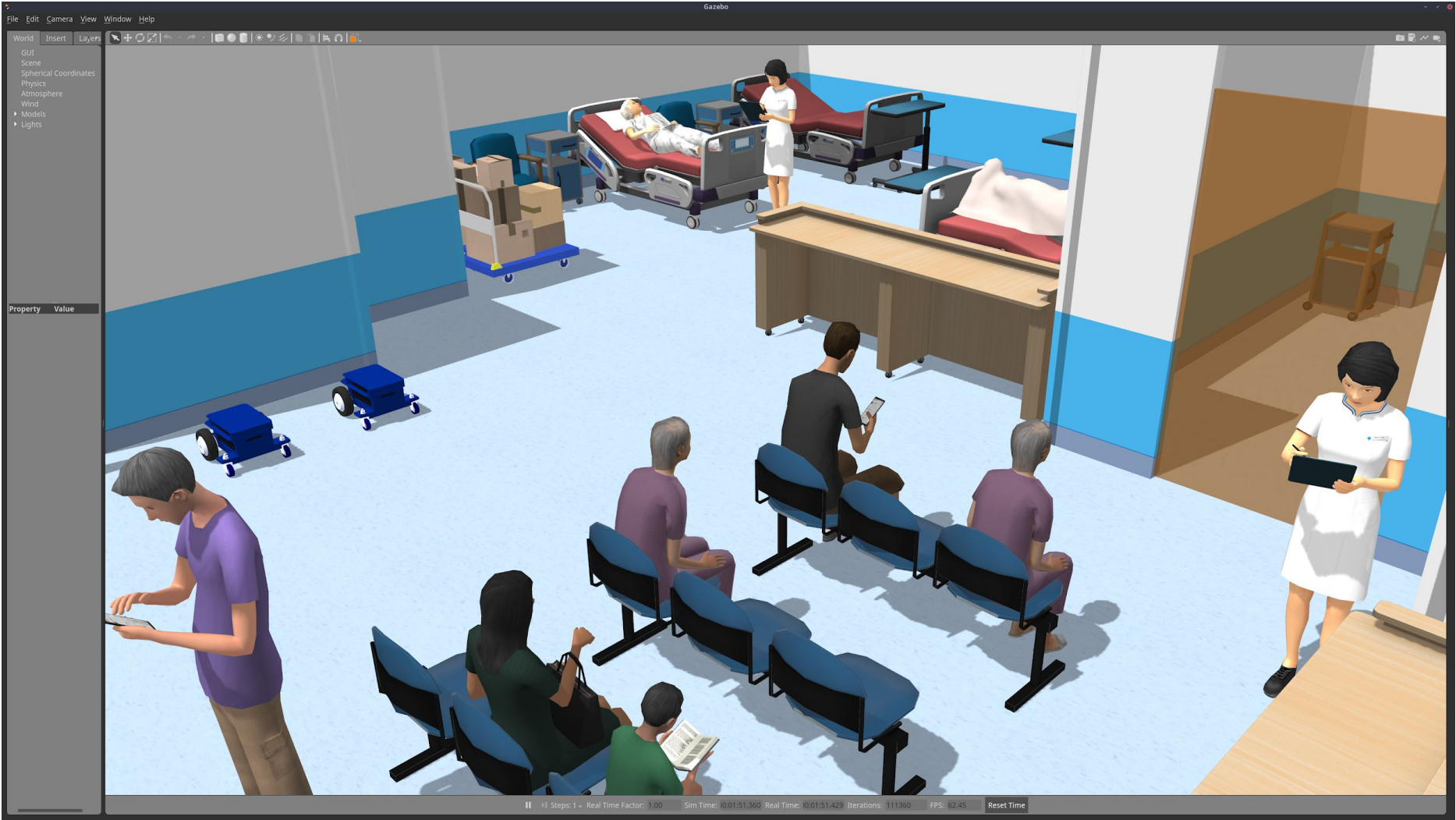
Generate Simulations

[rmf_building_map_tools](#) - a package with command line tools for processing your building files:

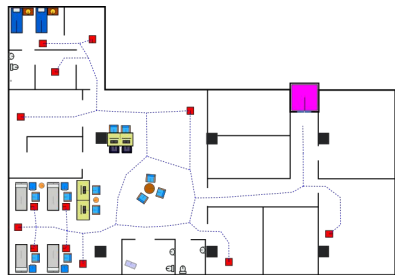
- `building_map_generator`: Produce gz or gazebo classic simulation worlds
- `building_map_generator nav`: Generate navigation graphs from your building file
- `building_map_model_downloader`: Fetch models from gz-fuel that are used in your building file

Use these tools in a CMakeLists.txt so they can be part of your `$ colcon build`!

- Working example in [rmf_demos_maps](#)



Traffic Editors: through the years

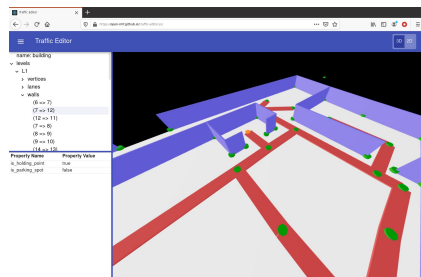


2018
Inkscape (SVG)
with annotations



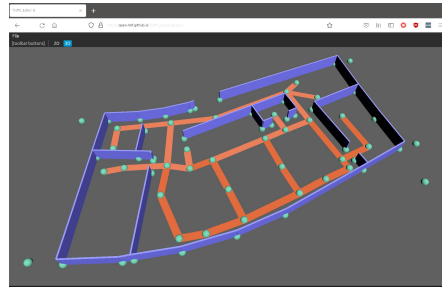
2019
C++ / Qt

desktop only



2021
TypeScript / THREE.js

web only



2022
Rust / Bevy

desktop and web

"classic" Qt version: https://github.com/open-rmf/rmf_traffic_editor

current Rust development: https://github.com/open-rmf/rmf_site



https://github.com/open-rmf/rmf_site

The RMF Site Editor

Next generation of RMF site editing

- Full 3D previews of the simulated facility, updated live as you edit it
- Runs natively on Linux, Windows, Mac, or web browsers
- Edit the placement of furniture, walls, doors, lifts, traffic lanes, human crowd behavior, cameras, and lights
- Easy to extend with custom plugins for application-specific features

Upcoming Features

- Preview camera feeds
- Save and link "read-only" portions, to reuse site information across different scenarios and allow manual additions to upstream data

Beyond

- A single application that can run on any computer (including Windows) to simulate a full deployment of RMF. All RMF nodes would run inside the site editor, running the same code as the real deployments. Users can add their own custom nodes as plugins.

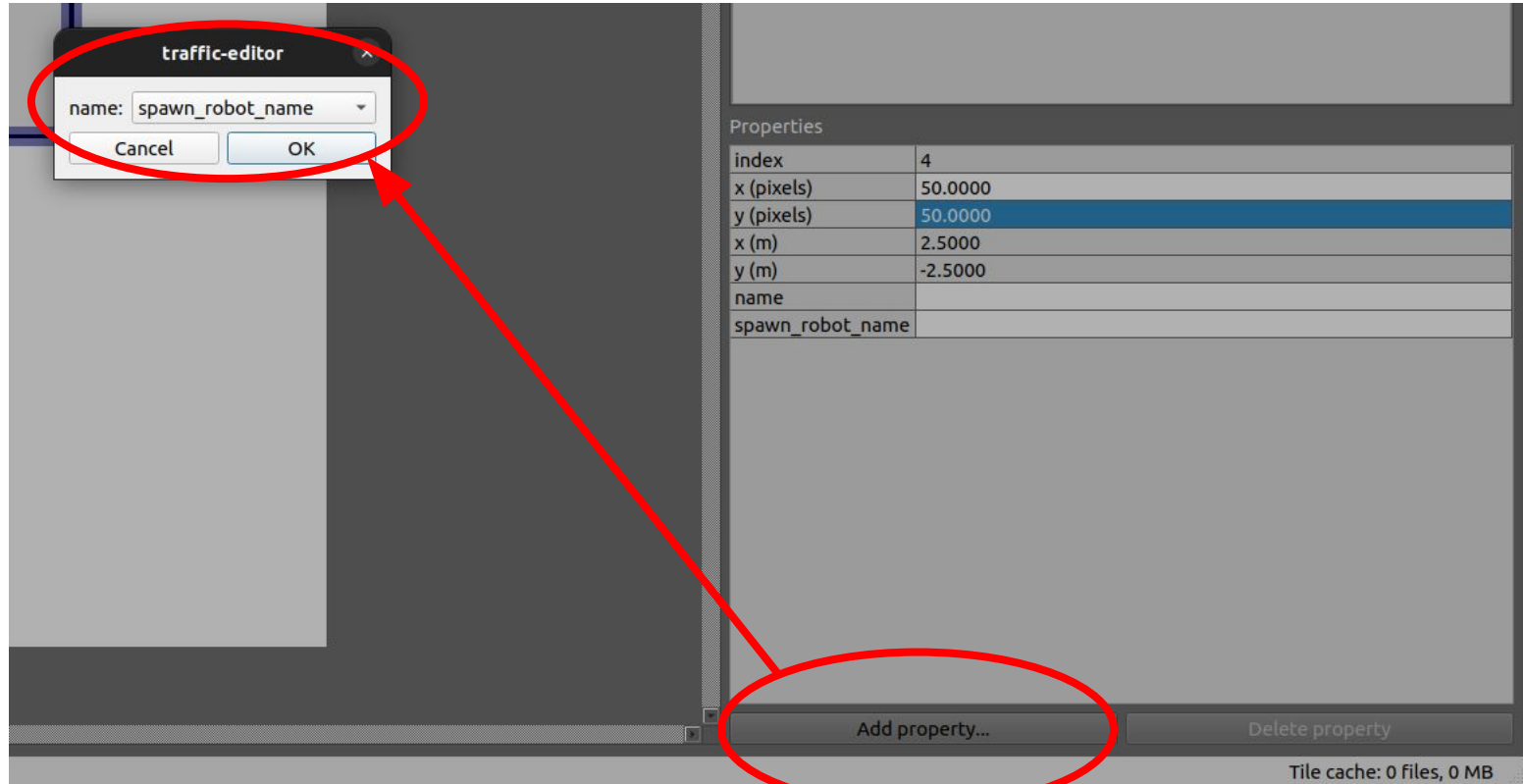
Try the WebGL version in your web browser!
https://open-rmf.github.io/rmf_site/



References

- [GitHub - open-rmf/rmf_traffic_editor](#)
- <https://osrf.github.io/ros2multirobotbook/traffic-editor.html>
- [RMF - using Traffic Editor to create floorplans](#)
- [Create an Environment with RMF Traffic-Editor](#)

Spawning a robot



Adding the vertex properties

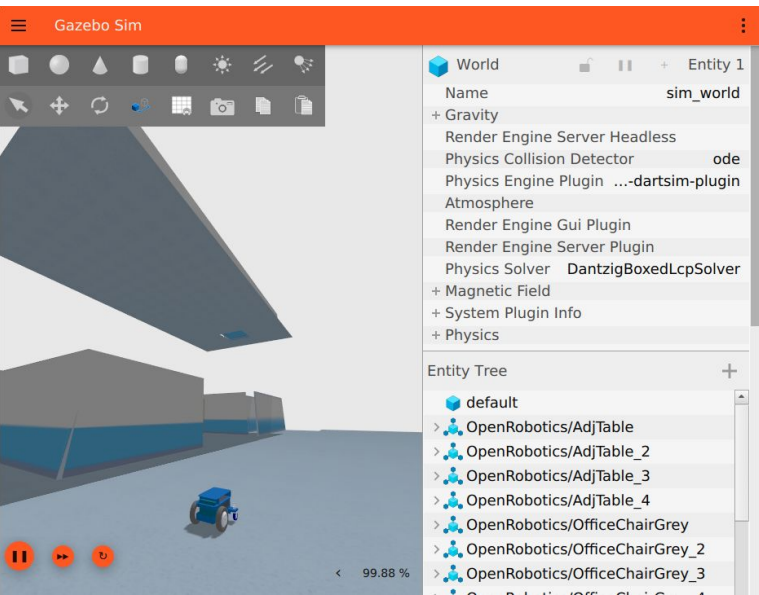
Properties

index	4
x (pixels)	50.0000
y (pixels)	50.0000
x (m)	2.5000
y (m)	-2.5000
name	tinyRobot1_charger
is_charger	true
spawn_robot_name	tinyRobot1
spawn_robot_type	TinyRobot

Configure the Gazebo model path

export

```
GZ_SIM_RESOURCE_PATH=`pwd`/icc_kyoto_world:`pwd`/models:/rmf_demos_ws/  
install/rmf_demos_assets/share/rmf_demos_assets/models
```



Don't worry about this
error for the moment

```
[ERROR] [1738239613.742456840] [slotcar_tinyRobot1]: Unable to determine the current level_name for robot [tinyRobot1].  
Kindly ensure the building_map_server is running. The RobotState message for this robot will not be published.
```